



using another claude account, i advanced till now, i.e completed

```
class DiscreteSignal
```

and LTISystem upto:

```
class LTISystem:
```

Show more

Verified implementations and prepared to advance toward response components

Checked both.

**DiscreteSignal**: matches everything we built together — good.

**output\_range**: correct.

```
python
```

```
def output_range(self, input_signal):
    return (input_signal.start_time + self.impulse_response.start_time,
            input_signal.end_time + self.impulse_response.end_time)
```

This matches `nx_min + nh_min` to `nx_max + nh_max` from the spec exactly.

**Step 11:** **get\_response\_components(input\_signal)**

**Purpose:** this is the "superposition" half of the assignment. For every nonzero input sample `x[k]`, it contributes a whole shifted-and-scaled copy of `h[n]`: the component `x[k] * h[n - k]`.

**Design:**

- Use `input_signal.nonzero_samples()` to get only the `(k, x_k)` pairs that matter — skips wasted work on zeros.
- For each `(k, x_k)`, the component signal is: take `self.impulse_response`, `shift(k)` it (so it's now `h[n-k]`), then `multiply(x_k)` (scales it by the input sample).
- Return a list of `(k, component_signal)` tuples — the spec wants `k` kept around too (useful later for reporting/debugging), not just the signals.

Try it — what does the loop look like?

python

```
def get_response_components(self, input_signal):
    components = []
    for k, x_k in input_signal.nonzero_samples():
        # your line here
    return components
```

fix this

```
def get_response_components(self, input_signal):
    components = []
    for k, x_k in input_signal.nonzero_samples():
        components.append(k,
            (input_signal.shift(k)).multiply(x_k))
    return components
```

Two bugs:

1. **Wrong signal being shifted.** You wrote `input_signal.shift(k)` — that shifts the *input* signal by its own sample time, which is meaningless here. You need to shift `self.impulse_response` by `k`, giving `h[n-k]`.
2. **Tuple syntax.** `components.append(k, (...))` passes two arguments to `.append()`, which only takes one. You need to wrap them together as a single tuple: `components.append((k, ...))`.

Fixed:

python

```
def get_response_components(self, input_signal):
    components = []
    for k, x_k in input_signal.nonzero_samples():
        component = self.impulse_response.shift(k).multiply(x_k)
        components.append((k, component))
    return components
```

Same logic, just fixed which signal gets shifted and correctly wrapped into a tuple before appending.

Want to move to `output_by_superposition` next? It's short now that this exists — just adds up all the components.

next

## Step 12: `output_by_superposition(input_signal)`

**Purpose:** add up every component response from `get_response_components` into one final output signal.

### Design:

- Start with an empty signal over the full output range — use `output_range(input_signal)` you already wrote, and `DiscreteSignal(start, end)` (all zeros).
- Loop over `get_response_components(input_signal)`, and `add()` each component onto your running total.
- Remember: `add()` returns a *new* signal rather than mutating — so you need to reassign your accumulator each time.

Skeleton — fill in the blanks:

```
python

def output_by_superposition(self, input_signal):
    start, end = self.output_range(input_signal)
    total = DiscreteSignal(start, end)
    for k, component in self.get_response_components(input_signal):
        total = ___
    return total
```

What goes in that blank?

```
total += self.impulse_response + k*component
```

Identified operator overloading issues and redundant multiplication errors

Not quite — a few issues here:

- `+=` doesn't work.** Your `DiscreteSignal` class has no `__add__` method defined, so Python doesn't know how to use `+` or `+=` on it. You have to call the method explicitly: `total.add(component)`.
- `k*component` is wrong math.** `component` here is already the fully-formed term `x[k] * h[n-k]` — that's exactly what `get_response_components` built for you in the last step. Multiplying it by `k` again would scale it by the *time index*, which means nothing physically. You just want to add the component as-is.
- `self.impulse_response` shouldn't appear here at all.** You already used the impulse response inside `get_response_components` to build each component. Adding it again here would double-count it incorrectly.

The correct line is simply:

```
python
```

```
total = total.add(component)
```

Full method:

python

```
def output_by_superposition(self, input_signal):
    start, end = self.output_range(input_signal)
    total = DiscreteSignal(start, end)
    for k, component in self.get_response_components(input_signal):
        total = total.add(component)
    return total
```

That's the superposition half of convolution done — 22/45 marks worth of `LTISystem` locked in (constructor, `output_range`, `get_response_components`, `output_by_superposition`).

Next up is the other half: `get_contributions_at_time` and `output_at_time` — the direct sliding-sum approach. Want to continue?

next

Mobilized momentum toward subsequent action

**Step 13:** `get_contributions_at_time(input_signal, n)`

**The math:** for a single output index `n`,  $y[n] = \sum_k x[k] \cdot h[n-k]$ . Most terms are zero (either `x[k]=0` or `h[n-k]=0`), so we only want the nonzero contributing terms.

**Design:**

- Loop over `input_signal.nonzero_samples()` to get each `(k, x_k)` with `x_k != 0`.
- For each `k`, compute `h[n-k]` via `self.impulse_response.get_value_at_time(n - k)`.
- If that value is also nonzero (use a tolerance, or just check `!= 0`), the term contributes. Compute `product = x_k * h_val`.
- Return a list of `(k, x_k, h_val, product)` tuples — matching the spec's example format exactly.

Skeleton:

python

```
def get_contributions_at_time(self, input_signal, n):
    contributions = []
    for k, x_k in input_signal.nonzero_samples():
        h_val = self.impulse_response.get_value_at_time(n - k)
        if h_val != 0:
            contributions.append((k, x_k, h_val, x_k * h_val))
    return contributions
```

Take a look at this one — do you see why we loop over `input_signal.nonzero_samples()` rather than `self.impulse_response.nonzero_samples()`? (Either would technically work since multiplication is commutative, but one is more natural given how the rest of your code is built.) Try `output_at_time` next — it should be almost free once you have this:

```
python

def output_at_time(self, input_signal, n):
    # your code here
```

check

```
# Arguments: input_signal is a DiscreteSignal and n is one
# output time index.
# Returns: float, the convolution-sum value y[n].
# Example: output_at_time(x, 4) returns the scalar sample
y[4]
```

Show more

Several bugs in both methods.

**output\_at\_time:**

1. **Wrong object.** `get_contributions_at_time` is a method on `LTISystem` (self), not on `self.impulse_response` (a `DiscreteSignal`). It doesn't exist there.
2. **Unpacking a list, not a tuple.** `get_contributions_at_time` returns a *list* of tuples (could be 0, 1, or many contributions) — you can't unpack it straight into `k, h_k, h_val, prod`. You need to loop over it and sum the products.

Fixed:

```
python

def output_at_time(self, input_signal, n):
    contributions = self.get_contributions_at_time(input_signal, n)
    return sum(product for k, x_k, h_val, product in contributions)
```

**output:**

1. **Wrong object again.** `output_range` is on `self`, not `self.impulse_response`.
2. **`DiscreteSignal(...)` takes two args, not a tuple.** `output_range` returns `(start, end)` — you need to unpack it: `DiscreteSignal(start, end)`, not `DiscreteSignal((start, end))`.
3. **`output_at_time` called on wrong object.** Same issue — it's `self.output_at_time(...)`, not `self.impulse_response.output_at_time(...)`.

Fixed:

```
python

def output(self, input_signal):
    start, end = self.output_range(input_signal)
    result = DiscreteSignal(start, end)
    for n in result.times():
        result.set_value_at_time(n, self.output_at_time(input_signal, n))
    return result
```

That's all 45 marks of `LTISystem` done. Next is `main.py` — `make_signal`, `signal_from_samples`, the impulse-response factory functions, and `max_absolute_difference`. Want to keep going?

keep going

Resolved to extend tutoring methodology forward

**Step 14:** `make_signal(start_time, end_time, values)`

**Design:** create a `DiscreteSignal` over the range, then drop each value in at its corresponding time. `values[0]` goes at `start_time`, `values[1]` at `start_time+1`, etc.

Skeleton:

```
python

def make_signal(start_time, end_time, values):
    signal = DiscreteSignal(start_time, end_time)
    for offset, value in enumerate(values):
        signal.set_value_at_time(start_time + offset, value)
    return signal
```

Try it, or if that looks right to you, tell me and we'll move to `signal_from_samples`.

check, correct me if wrong:

```
# Arguments: start_time and end_time are integers; values is a
            list of integers
```

Show more

Both have real bugs, and both are also missing `return result` (well, `signal_from_samples` has it, `make_signal` doesn't).

**`make_signal`:**

1. `for t, val in result.times(), values:` isn't valid Python — you can't iterate two separate sequences like that with a comma. You need `zip()` to pair them up.
2. Missing `return result` at the end.

Fixed:

python

```
def make_signal(start_time, end_time, values):
    result = DiscreteSignal(start_time, end_time)
    for t, val in zip(result.times(), values):
        result.set_value_at_time(t, val)
    return result
```

**`signal_from_samples`:**

1. `{tx, val}` isn't valid unpacking syntax — that's a set literal, not a way to destructure a pair.
2. The same `,` issue — can't iterate two sequences like that.
3. The whole approach is overcomplicated. Since `DiscreteSignal` already initializes every value to 0, you don't need an `if/else` at all — you only need to set the values that are specified in `samples`, and leave everything else at its default zero.
4. `if samples.get(tx) != 0 and t==tx` doesn't make sense — you're not actually looping over the dict correctly, and this condition doesn't match what you want (place a value only where the dict has an entry for that time).

Simplify it to just this:

python

```
def signal_from_samples(start_time, end_time, samples):
    result = DiscreteSignal(start_time, end_time)
    for t, val in samples.items():
        result.set_value_at_time(t, val)
    return result
```

Since the signal already starts all-zero, you only loop over the dict's actual entries — no need to touch every `t` in `result.times()` at all.

Try both again with these fixes in mind, or if you're good with these corrected versions, say so and we'll move to `impulse_identity` / `impulse_moving_average`.

but it said

```
# Arguments: start_time/end_time are integers; samples maps time
indices to values.
# Returns: DiscreteSignal over the full range, with unspecified
samples left as 0.
# Example: signal_from_samples(0, 3, {0: 1, 2: 5}) creates values
[1, 0, 5, 0].
```

so, i should handle where t is not available in samples

Reconciled default behavior with explicit handling requirements

Good instinct to double check, but you don't need special handling for that case — and here's why.

Look back at `DiscreteSignal.__init__`:

```
python

self.values = np.zeros(end_time - start_time + 1)
```

Every sample is **already 0 by default** the moment you create the signal. So for `signal_from_samples(0, 3, {0: 1, 2: 5})`:

- `result = DiscreteSignal(0, 3)` → values start as `[0, 0, 0, 0]`
- Loop over `samples.items()`: sets `t=0` → `1`, sets `t=2` → `5`
- `t=1` and `t=3` are never touched — they stay at their default `0`

Final result: `[1, 0, 5, 0]` — exactly matching the spec's example, with zero extra code needed.

The comment "with unspecified samples left as 0" is describing the *behavior*, not telling you to write an explicit branch for it — the zero-initialization from the constructor handles it for free. Explicitly looping over `result.times()` and checking membership in `samples` would work too, but it's more code to do the same thing the constructor already gives you.

So this version is complete and correct:

```
python

def signal_from_samples(start_time, end_time, samples):
    result = DiscreteSignal(start_time, end_time)
    for t, val in samples.items():
        result.set_value_at_time(t, val)
    return result
```

Ready to move to the impulse response factory functions ( `impulse_identity`, `impulse_moving_average`, etc.)?

yes

### Step 15: `impulse_identity()`

**Design:** the identity system just passes the input through unchanged: `h[0] = 1`, nothing else. That's a signal over the range `0..0` with a single value.

You already have `make_signal` for exactly this. Try it:

```
python

def impulse_identity():
    # your line here
```

Think about what `start_time`, `end_time`, and `values` should be, given the range is just `0..0`.

```
def impulse_identity():
    result = DiscreteSignal(0,0)
    result.set_value_at_time(0,1)
    return result
```

Show more

Identified off-by-one range bug in moving average function

`impulse_identity` — correct. Clean and matches the spec exactly.

`impulse_moving_average` — off-by-one bug:

`DiscreteSignal(0, length)` creates a range `0..length` inclusive, which is `length + 1` samples, not `length`. For `impulse_moving_average(3)`, you want exactly `h[0]`, `h[1]`, `h[2]` — that's range `0..2`, i.e. `end_time = length - 1`.

Fixed:

```
python
```

```
def impulse_moving_average(length):
    result = DiscreteSignal(0, length - 1)
    for t in result.times():
        result.set_value_at_time(t, 1 / length)
    return result
```

Quick check: with your buggy version, `impulse_moving_average(3)` would create `h[0]=h[1]=h[2]=h[3]=1/3` — four terms instead of three, each still weighted `1/3`, so the weights wouldn't even sum to 1. That's the kind of subtle bug that wouldn't crash but would quietly give wrong convolution outputs.

**Step 16:** `impulse_weighted_smoothing()` and `impulse_first_difference()`

Both are just fixed 2-3 tap filters — use `signal_from_samples` (cleaner than `make_signal` here since the values aren't contiguous-only, though either works):

- Weighted smoothing: `h[0]=0.5, h[1]=0.3, h[2]=0.2`
- First difference: `h[0]=1, h[1]=-1`

Try both.

what to pass inside `signal_from_samples()`

`signal_from_samples(start_time, end_time, samples)` — you need the range and a dict mapping each time index to its value.

For weighted smoothing, the range is `0..2` and the dict is `{0: 0.5, 1: 0.3, 2: 0.2}`:

```
python

def impulse_weighted_smoothing():
    return signal_from_samples(0, 2, {0: 0.5, 1: 0.3, 2: 0.2})
```

Now try `impulse_first_difference()` yourself — what range and dict does `h[0]=1, h[1]=-1` need?

```
def impulse_first_difference():
    return signal_from_samples(0,1,{0:1, 1:-1})
```

Correct — range `0..1`, dict `{0: 1, 1: -1}` matches `h[0]=1, h[1]=-1` exactly.

That's all six impulse response factories done. Last piece of graded logic left:

`max_absolute_difference(first_signal, second_signal)`.

**Design:** loop over the *combined* range of both signals (same "union" idea as `add`), track the largest `abs(diff)` seen.

Try it:

python

```
def max_absolute_difference(first_signal, second_signal):
    start = min(first_signal.start_time, second_signal.start_time)
    end = max(first_signal.end_time, second_signal.end_time)
    # your loop here
```

```
def max_absolute_difference(first_signal, second_signal):
    start = min(first_signal.start_time,
second_signal.start_time)
    end = max(first_signal.end_time, second_signal.end_time)
    max=0
    for t in range(start, end+1):
        diff = abs(first_signal.get_value_at_time(t) -
second_signal.get_value_at_time(t))
        if diff > max:
            max = diff
    return max
```

Logic is correct, but one real problem: **you named your variable `max`**, which shadows Python's built-in `max()` function for the rest of this function's scope. It happens to still work here since you don't call `max(...)` anywhere else in this function, but it's a bug waiting to happen — if you ever add another comparison in this function, `max()` the builtin will silently be gone, replaced by your float variable, causing a confusing `TypeError`.

Rename it:

python

```
def max_absolute_difference(first_signal, second_signal):
    start = min(first_signal.start_time, second_signal.start_time)
    end = max(first_signal.end_time, second_signal.end_time)
    largest = 0
    for t in range(start, end + 1):
        diff = abs(first_signal.get_value_at_time(t) - second_signal.get_value_at_time(t))
        if diff > largest:
            largest = diff
    return largest
```

Same behavior, no shadowing risk.

That's every `NotImplementedError` in both files done — `signal_lti.py` fully implemented, and `main.py`'s five helper functions (`make_signal`, `signal_from_samples`,

the four impulse factories, `max_absolute_difference`) all complete.

What's left before you can submit:

1. **Test it end-to-end** — run `python3 main.py inputs/1.txt --out-dir outputs/1` and check for crashes/wrong output.
2. **Create your four `inputs/*.txt` files** (small+custom, small+built-in, long+custom, long+built-in) — the spec gives you the exact format.
3. **Write `cmd.txt`** with the four run commands.
4. **Zip it up** as `<student_id>.zip` per the submission structure.

Want help with any of those next — e.g. drafting the input files, or debugging once you run it?

